Levenberg-Marquardt Algorithm-based AMCL Parameter Optimization on an Experimental Global Localization Scenario

Önder İhsan Kul

29.04.2026

1 Introduction

Robot localization proves to be a crucial field of study in mobile robotics applications. This study focuses on performance analysis and parameter optimization of the AMCL algorithm on the global localization problem through a test scenario.

In the test scenario, the robot starts at a random location of a previously known 500x500 cm map. Then the robot is manually operated to navigate to 3 different waypoints on the map, at each waypoint the robot is stopped for 2 seconds and operated to the next waypoint. After completing this traversal twice, the test is completed.

Global localization performance was assessed by comparing robot position estimates on the waypoints. Depiction of the problem is given in the following Figure 1. After initial results are obtained using default parameters, a parameter set of 9 parameters that quantify sensor and motion models' noise are selected, then the error minimization is modeled as an optimization problem. Details for the methodology are described in section 2, the report is concluded in section 3.

2 Methodology

2.1 Filtered Odometry

In this section, filtered odometry calculation system will be explained. First, we provide a flow chart for the whole system, then explain each component by its mathematical model, and contribution to the system. Block diagram for the system can be seen at Figure 2.

Rear-axle Bicycle Model For the kinematic modeling of the vehicle, rear-axle centered bicycle model is used, drawing for the model can be found at Figure 3, state equations for the model are provided at Equation 1.

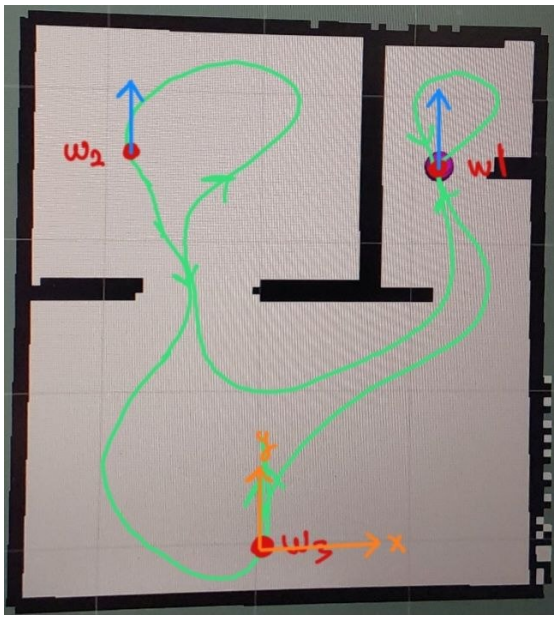


Figure 1: Depiction of the Experimental Setup

$$\begin{aligned} \dot{x} &= vcos(\theta) \\ \dot{y} &= vsin(\theta) \\ \dot{\theta} &= \frac{vtan(\delta)}{L} \\ \dot{\delta} &= \Phi \end{aligned} \tag{1}$$

Bicycle Odometry Model For odometry calculations, "VESC Node" calculates velocity and steering angle using a linear function of motor speed state and a linear function of motor steering state, which are calibrated experimentally.

After velocity and steering angle are obtained, odometry is calculated using the discrete state model given in Equation 2.

$$\begin{aligned} x_0 = 0, y_0 = 0, \theta_0 = 0 \\ x_{k+1} = x_k + v_k cos(\theta)\Delta t \\ y_{k+1} = y_k + v_k sin(\theta)\Delta t \\ \theta_{k+1} = \theta_k + \frac{v_k tan(\delta_k)}{L}\Delta t \end{aligned} \tag{2}$$

Odometry Message The odometry message published by "VESC Node" includes velocity relative to robots base frame, pose estimates in the odom

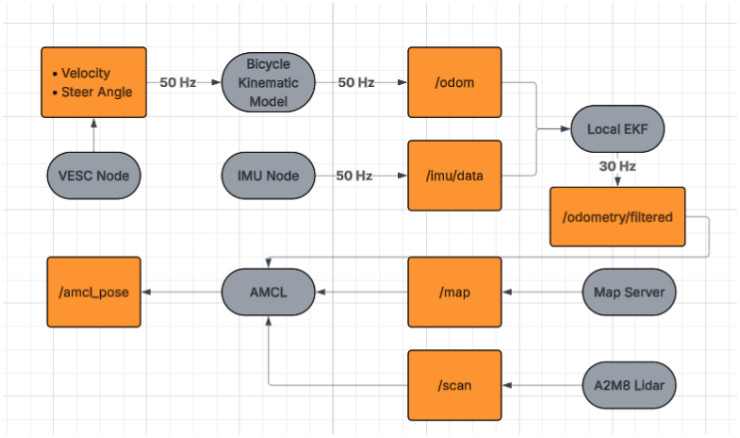


Figure 2: Odometry System Block Diagram

frame and angular velocity estimate. This odometry message is published to the */odometry/filtered* topic with data rate of 50 Hz, using the relations given in Equation 3

$$\begin{aligned} pose.x &= x_k \\ pose.y &= y_k \\ twist.linear.x &= v_k \\ twist.angular.z &= \frac{v_k tan(\delta_k)}{L} \end{aligned} \tag{3}$$

2.2 Robot Localization EKF

For robust position estimate, EKF algorithm is utilized in the robot. Algorithm is implemented with the commonly used ROS2 package *robot localization*. Details of the algorithm implementation in the *robot localization* package is described in this section.

Motion Model Generally, Runga-Kutta methods are used to approximate the solution for an Initial Value Problem(IVP) formulated in Equation 4

$$\begin{aligned} y(t_0) &= y_0 \\ \dot{y}(t) &= f(y(t), t) \end{aligned} \tag{4}$$

If we take f(y) as the derivative function for $x_{k+1} = (x, y, \theta, \dot{x}, \dot{y})^T$, then applying the RK4 method for x_{k+1} will yield a good approximation for estimates of x, y.

In the package, x, y estimates are computed using the RK4 approximation to prevent discrete jumps in the state estimate, with the steps provided in Equation

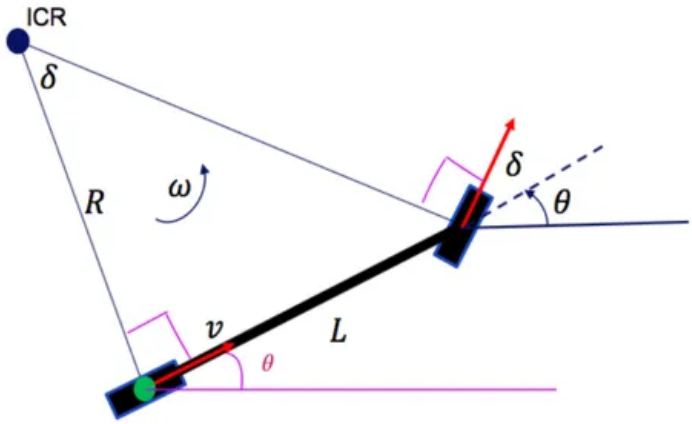


Figure 3: Rear-centered Bicycle Model

5.

$$
\begin{aligned}
k_1 &= f(x_k) \\
k_2 &= f(x_k + \frac{k_1 \Delta t}{2}) \\
k_3 &= f(x_k + \frac{k_2 \Delta t}{2}) \\
k_4 &= f(x_k + k_4 \Delta t) \\
x_{k+1} &\approx x_k + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k4);
\end{aligned} \tag{5}
$$

Here, the derivative function $f(x, y, \theta, \dot{x}, \dot{y})$ can be simply written as:

$$
\begin{aligned}
\dot{x} &= v_x cos(\theta) - v_y sin(\theta) \\
\dot{y} &= v_x sin(\theta) + v_y cos(\theta) \\
\dot{\theta} &= \omega \\
\ddot{x} &= a_x \\
\ddot{y} &= a_y
\end{aligned} \tag{6}
$$

Overall, state transition equations for the 8D state vector can be written as:

$$
\begin{aligned}
x_{k+1} &= x_k + RK4_x \\
y_{k+1} &= y_k + RK4_y \\
\theta_{k+1} &= \theta_k + \omega \Delta t \\
v_{x,k+1} &= v_{x,k} + a_{x,k} \Delta t \\
v_{y,k+1} &= v_{y,k} + a_{x,k} \Delta t \\
\omega_{k+1} &= \omega_k \\
a_{x,k+1} &= a_{x,k} \\
a_{y,k+1} &= a_{y,k}
\end{aligned} \tag{7}
$$

Note that in this model, global x and y coordinates are used as state variables whereas velocity and acceleration states are entirely in the robots local plane. Remaining parts of the implementation such as measurement update, covariance prediction and covariance update are coherent with the original EKF implementation, so we omit them here.

In the *robot localization package*, each sensor input data is taken as a measurement update. Whenever a new sensor data comes, the system calculates time difference, makes a prediction and does measurement update with the sensor data. H matrices for each sensor data is computed from the .yaml configuration file. Jacobian matrices for the covariance update, Kalman gain calculation and measurement update are done with the Euler model. This enables the algorithm to generate a robust position estimate for sensor streams with different frequencies.

In our implementation, (v_x, v_y, ω) measurements from odometry and (a_x, a_y, ω) measurements from IMU are used.

Lastly, calculated odometry message is published to the /odometry/filtered topic with the data rate of 30 Hz.

2.3 Problem Formulation as Non-linear Least Squares

Goal of the System The goal of the system described in the Figure 2 is to make state estimates at given waypoints as close to the ground truth robot positions as possible.

Least Squares Form Ground truth values can be described as an 18D vector, with x, y, θ estimates for each of the 6 waypoints. In our approach, AMCL x, y, θ estimates are assumed to be a differentiable function of a selected set of AMCL parameters.

Given the ground truth values for x, y, θ estimates, our goal can be viewed as minimizing the sum of squared residuals of a function, we can formulate the problem as a Non-linear least squares problem. In this problem, f is a function of a selected set of parameters and the residual vector is an 18D vector defined by differences betweem ground truth values and position estimates. Problem formulation can be found in Equation 8

$$\begin{aligned} \hat{y}_i &= f(x_i, \beta) \\ \beta &= (\beta_1, \beta_2, \ldots, \beta_n) \\ (x_1, y_1), (x_2, y_2) \ldots (x_m, y_m), m > n \\ r_i &= y_i - f(x_i, \beta) \\ S &= \sum_{i=1}^{m} ||r_i||^2 \end{aligned} \tag{8}$$

Parameter Selection In this approach, global position estimates of the robot in different waypoints are modeled as a differentiable function of the tun-

able parameters of the AMCL algorithm. A set of 9 parameters quantifying sensor and odometry models' noise are selected to minimize the position estimation errors.

In this subsection, we explain the meaning of and the reason why each parameter is selected for the optimization problem.

First 4 parameters, are selected from the noise parameters in the odometry motion model. Odometry motion model models the noise in the odometry calculations with 4 variance parameters. AMCL algorithm updates the probability distribution of $p(x_t|x_{t-1}, u_t)$ using subsequent odometry measurements. Pseudo for its calculation is given in the Figure 4

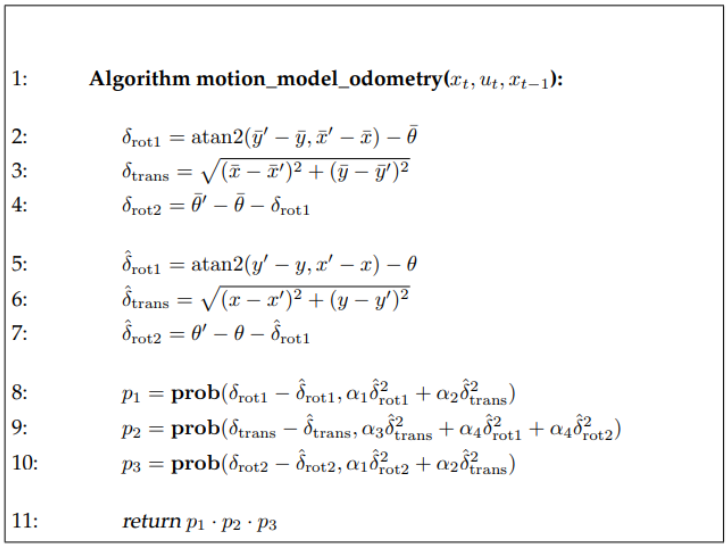


Figure 4: Odometry Motion Model

5th parameter is the expected measurement weight used in the scan sensor model. This model represents the expected frequency of true laser measurements, which implicitly determines the measurement update $p(z_t|x_t)$. Random measurement weight is assigned as $p(rand) = 1 - p(expected)$, quantifying random measurement frequency.

6,7 and 8th parameters are minimum difference requirements for AMCL update to take place. These parameters balance the use of computational resources by not updating state estimates for little changes in the odometry estimates. Besides, they prevent particle updates that would be done with noisy odometry measurements small magnitude of change.

9th parameter quantifies the trust in the lidar sensor measurements in the likelihood field model. This model matches closest occupied cell in the map with lidar sensor estimate for each ray and assigns a probability to the sensor reading using the variance specified in this parameter. A higher value means sensor measurements are thought to be noisy, a lower value means the sensor is

believed to make more precise measurements.

2.4 Parameter Optimization

We followed a 2-step approach for the parameter optimization process. First, a good starting point is found using random search on reasonable parameter intervals. These intervals must be selected carefully to force the algorithm to find a plausible starting point. Otherwise, the algorithm can end up in a point far away from local minima, which will make the optimization process much harder.

Random Parameter Search As mentioned above, parameter intervals must be selected carefully to obtain a reasonable starting point for other optimization algorithms. For our problem, selected parameter intervals for random search is given in the Table 1. In our implementation, 100 iterations on random parameters is used to obtain a starting point.

Parameter	Start	End
α_1	0.03	0.25
α_2	0.03	0.25
α_3	0.03	0.25
α_4	0.03	0.25
w_{mea}	0.1	0.4
min_x	0.05	0.3
min_y	0.05	0.3
min_θ	0.1	0.3
σ_{hit}	0.1	0.4

Table 1: Random Parameter Search Intervals

Best configuration found after Random Search and MAE and RMSE scores for that configuration are given in the Table 2 and Table 3.

Parameter	Value
α_1	0.1307
α_2	0.0589
α_3	0.1713
α_4	0.2097
w_{mea}	0.8089
min_x	0.1286
min_y	0.2200
min_θ	0.2068
σ_{hit}	0.3574

Table 2: Best Configuration After Random Search

	x	y	θ	Total
RMSE	-	-	-	0.554
MAE	0.1741	0.1675	0.1154	0.457

Table 3: Random Search Error Metrics

Levenberg-Marquardt Algorithm After obtaining an initial point for the parameter search, we used Levenberg-Marquardt Algorithm to further minimize the error metrics.

Normally, under the linear approximation assumption, non-linear least squares problem can be solved iteratively using the Gauss-Newton method, the approximation and a solution step is given in the Equation 9

$$\begin{aligned} f(x_k, \beta) &\approx f(x_k, \beta_k) + Df(x_k, \beta_k)(x - x_k) \\ \beta_{k+1} &= \beta_k + (J_r^T J_r)^{-1} J_r^T r(\beta_k) \end{aligned} \tag{9}$$

Gauss-Newton method guarantees to converge to a solution when the Hessian matrix is positive-definite. Levenberg-Marquardt method combines the features of Steepest Descent and Gauss Newton by adding a dampening term and scaling by diagonals of the Jacobian matrix to account for curvature.

Levenberg-Marquardt solution for the non-linear least squares problem is given in the Equation 10

$$\beta_{k+1} = \beta_k + (J_r^T J_r + \lambda diag(J_r^T J_r))^{-1} J_r^T r(\beta_k) \tag{10}$$

In the latest implementations of Levenberg Marquardt method, lambda value is adjusted depending on whether the residual decreases or increases. A decreasing residual means the model behaves more similar to Gauss-Newton approach, because we see convergence. If it increases, the model behaves more cautiously and increases the lambda to behave like steepest descent.

Lambda values and RMSE values corresponding to them are given in the Table 4

Iter	RMSE	Lambda
1	0.25	2
2	0.1997	8
3	0.1767	4
4	0.1687	1.6×10^2
5	0.1555	6.55×10^5

Table 4: LMA RMSE on each iteration

2.5 LMA Performance Metrics

Final RMSE and MAE values are given in Table 5

	x	y	θ	Total
RMSE	-	-	-	0.1555
MAE	0.1597	0.0746	0.136	0.3703

Table 5: LMA Error metrics

3 Conclusion

In this report, we tested and analyzed amcl performance on real-life global localization scenario. Then we selected a set of parameters to minimize the global localization error. First, we applied random search on carefully selected parameter intervals to have a reasonable starting point for further optimization. Then, we ran Levenberg-Marquardt Algorithm with the initial point as the point found after random search.

If we compare the results, Random Search yields an RMSE of 0.554 while LMA yields 0.1555. For MAE, Random search yields 0.457 total MAE whereas LMA yields MAE of 0.3703. Significant difference in the RMSE measurement means LMA yields close estimates in all the waypoints. On the contrary, A high RMSE in the random search with closer MAE to LMA means random search parameter set makes close estimates in some waypoints while overshooting in others.

In conclusion, LMA Optimization proves to be robust approach for AMCL parameter optimization. In future reports, we'll analyze other parameter optimization algorithms for different scenarios.